

Software Design Patterns: Concepts and Foundations

Study notes for Ziyadinkhan Kudaibergenuly | SE2501

Examples: RideReady vehicle rental project

1. What is a design pattern?

A design pattern is a named, reusable approach to a recurring software design problem. It describes responsibilities and relationships between objects. It is not a ready-made library or code that must be copied exactly.

A pattern should solve an actual problem. Adding more interfaces and classes is not automatically better design. For a small, fixed program, direct construction with new may be simpler. Factories become useful when construction choices vary and several clients should be insulated from those choices.

Three common categories:

- Creational patterns organize object creation: Factory Method, Abstract Factory, Builder, Prototype, Singleton.
- Structural patterns organize how objects and classes fit together: Adapter, Decorator, Composite, Facade.
- Behavioral patterns organize collaboration and algorithms: Strategy, Observer, Command, State, Template Method.

Factory Method and Abstract Factory are creational patterns. The assignment asks you to understand the problem each solves, not merely recognize its class names.

2. Object-oriented foundations

Class and object

A class defines a type and its behavior. An object is an instance of that class. Car is a class; new Car() creates an object. Many distinct objects can have the same class.

Interface and contract

An interface describes operations that implementations provide. A useful contract also includes the meaning and expected behavior of those operations. Java checks method signatures; it does not automatically prove all behavioral expectations.

Vehicle declares description() and wheels(). A client can use these methods without knowing whether the object is a Car or Motorcycle.

Abstraction and encapsulation

Abstraction exposes the relevant idea while omitting unnecessary details. Vehicle represents the operations needed by the rental workflow.

Encapsulation keeps representation and rules behind a boundary. AccessoryRental keeps its product references private and validates them in the constructor. Other code cannot replace its fields with an invalid pair.

Abstraction answers “what can this object do?” Encapsulation controls “which details and changes can other code access?”

Inheritance and composition

Inheritance expresses an is-a relationship and permits overriding inherited operations. CarFactory is a VehicleFactory.

Composition, broadly understood as building behavior from other objects, expresses a has-a relationship. AccessoryRental holds safety and security objects. It receives a factory rather than inheriting from one.

Prefer composition when behavior should be replaceable without adding subclasses. Inheritance is appropriate when the subtype relationship and shared behavior are meaningful. This is a guideline, not a universal prohibition on inheritance.

Polymorphism and dynamic dispatch

Polymorphism lets the same interface operation have different behavior depending on the actual object.

```
VehicleFactory factory = new CarFactory();  
String message = factory.prepareRental();
```

The variable's declared type is VehicleFactory, while its object is CarFactory. The inherited prepareRental() calls createVehicle(). Java dispatches that call to CarFactory.createVehicle(), which returns a Car.

The left-hand type controls which operations the compiler permits. The runtime object determines which overridden instance-method implementation executes.

3. Dependencies, coupling, and cohesion

A depends on B when A needs B's type or behavior to do its work. Examples include a field type, method parameter, method call, or direct construction.

Coupling describes how strongly parts depend on one another. A client that constructs CarAlarm is tied to that concrete class. A client using SecurityDevice can accept another implementation of the same contract.

Cohesion describes how closely a component's responsibilities belong together. A factory focused on construction is more cohesive than a class that also reads console input, prints invoices, and stores customer records.

Low coupling does not mean no dependencies. The aim is to put dependencies at boundaries that are stable and useful.

Dependency injection means providing a dependency from outside. AccessoryRental receives an AccessoryFactory through its constructor. No framework is required.

The composition root is the place that selects and connects concrete implementations. In this project, Main performs that role. Concrete constructors must exist somewhere; patterns control where those choices are made.

4. Part A: Factory Method

Problem and solution

Problem: a reusable workflow needs an object, but different subclasses should decide which concrete object to create.

Solution: the creator declares an overridable factory method that returns a product abstraction. Concrete creators override it. The shared workflow calls that method rather than directly constructing a concrete product.

Roles in RideReady

- Product: Vehicle.
- Concrete Products: Motorcycle and Car.
- Creator: VehicleFactory.
- Factory Method: createVehicle().
- Concrete Creators: MotorcycleFactory and CarFactory.
- Shared workflow: VehicleFactory.prepareRental().

```
public abstract class VehicleFactory {  
    public abstract Vehicle createVehicle();  
  
    public final String prepareRental() {  
        Vehicle vehicle = createVehicle();  
        return "Ready: " + vehicle.description()  
            + " (" + vehicle.wheels() + " wheels)";  
    }  
}
```

The excerpt is formatted across shorter lines but has the same behavior as the project.

Execution trace

1. Main creates CarFactory and passes it through a VehicleFactory parameter.
2. The demo calls prepareRental().
3. prepareRental() invokes createVehicle().
4. Dynamic dispatch chooses CarFactory.createVehicle().
5. That method constructs Car and returns it as Vehicle.
6. The shared workflow calls the Vehicle methods and produces the rental description.

prepareRental() is final, so subclasses cannot replace that workflow. createVehicle() is abstract, so each concrete creator must provide it. This combination keeps the workflow stable while varying creation.

Factory Method versus Simple Factory

A Simple Factory usually centralizes creation in an ordinary helper, often selecting a type from an argument. It does not require a creator subclass hierarchy.

A method being named create or factory does not by itself make it the Factory Method pattern. The key feature here is subclass-controlled creation used through the creator abstraction.

Static factory methods such as a named alternative to a constructor are also a separate idea. “Factory” is a broad word; explain the actual structure.

5. Part B: Abstract Factory

Problem and solution

Problem: a client needs multiple related product types, and the selected implementations should form a consistent family.

Solution: give the client a factory interface with one creation operation for each product type. Each concrete factory supplies one complete family.

Roles in RideReady

- Abstract Factory: AccessoryFactory.
- Abstract Products: SafetyEquipment and SecurityDevice.
- Motorcycle factory: MotorcycleAccessoryFactory.
- Motorcycle products: Helmet and DiscLock.
- Car factory: CarAccessoryFactory.
- Car products: ChildSeat and CarAlarm.
- Client: AccessoryRental.

Think of product types as columns and families as rows:

- SafetyEquipment column: Helmet or ChildSeat.
- SecurityDevice column: DiscLock or CarAlarm.
- Motorcycle row: Helmet plus DiscLock.
- Car row: ChildSeat plus CarAlarm.

```
public interface AccessoryFactory {
    SafetyEquipment createSafetyEquipment();
    SecurityDevice createSecurityDevice();
}
```

Execution trace

1. Main supplies a CarAccessoryFactory to AccessoryRental.
2. The client calls createSafetyEquipment() through AccessoryFactory.
3. The concrete factory returns ChildSeat as SafetyEquipment.
4. The client calls createSecurityDevice().
5. The concrete factory returns CarAlarm as SecurityDevice.
6. The client validates the pair and works through the product interfaces.

The client does not instantiate ChildSeat or CarAlarm. Changing the factory changes the family without adding a product-selection branch to the client.

What consistency means here

Both products expose category(). The client rejects different categories. That is a runtime invariant check, not compile-time proof of compatibility.

The model checks matching family labels, not real-world physical compatibility. It also does not automatically ensure that a separately supplied vehicle factory matches the accessory factory. Main wires those matching choices.

Do not claim that Abstract Factory makes every invalid combination impossible. Its guarantees depend on the contracts and implementations.

6. Comparing the two patterns

Factory Method:

- Main variation: which concrete product a creator subclass constructs.
- Main mechanism: inheritance and overriding.
- RideReady example: CarFactory overrides createVehicle().
- Typical use: preserve a common workflow while subclasses choose a product.

Abstract Factory:

- Main variation: which family of related products a factory supplies.
- Main mechanism: a factory object with several product-creation operations.
- RideReady example: CarAccessoryFactory supplies a safety product and a security product.
- Typical use: switch a whole family while the client stays on interfaces.

“One object versus several objects” is a useful first approximation, but it is not the entire definition. Focus on subclass-based creation versus the family-creation interface. The patterns can also be combined.

Adding a vehicle type requires a new Vehicle and VehicleFactory subclass. Adding an accessory family requires implementations of both product types and a new factory. This project's category enum must also be extended.

Adding an entirely new accessory product type requires a new factory operation and changes to every concrete factory. Abstract Factory makes new families easier, but makes new product types more expensive.

7. Principles to know instead of “basic theorems”

These are design principles and heuristics. They are not mathematical theorems that prove a design is always correct.

Single Responsibility Principle (SRP)

A module should have one coherent responsibility or reason to change. A factory constructs its products; Main handles application wiring and console presentation. This does not mean every class must contain only one method.

Open/Closed Principle (OCP)

Aim to extend behavior through stable abstractions without repeatedly modifying existing client logic. A new VehicleFactory subclass can reuse prepareRental(). The category enum still needs editing for new accessory families, so this implementation is not completely closed to modification.

Liskov Substitution Principle (LSP)

Subtypes should be usable wherever their base abstraction is expected without violating the promised behavior. A Vehicle implementation should return a meaningful description and wheel count. Signature compatibility alone is insufficient.

For factories, returning null or mismatched products violates the client's expectations. The client detects several such violations, but validation is not a substitute for correct implementations.

Interface Segregation Principle (ISP)

Clients should not depend on operations they do not need. `SafetyEquipment` and `SecurityDevice` have separate contracts. A single enormous interface for every vehicle and accessory operation would force unrelated dependencies.

A small project may not show a strong practical need for this principle; do not invent requirements solely to demonstrate it.

Dependency Inversion Principle (DIP)

High-level policy and low-level implementations should depend on suitable abstractions. `AccessoryRental`'s policy uses `AccessoryFactory` and product interfaces, while concrete classes implement those interfaces.

DIP is a design principle. Dependency injection is one technique that can support it. They are related but are not synonyms.

DRY, KISS, and YAGNI

DRY: avoid duplicating knowledge or rules. Validate the accessory-pair invariant in one place.

KISS: keep the solution understandable. A console demonstration makes the patterns visible.

YAGNI: do not add features merely because they might be needed later. A database and web server are unnecessary for this assignment.

These principles can pull in different directions. Factories introduce extra classes; use the added structure only where its flexibility or teaching purpose justifies the cost.

8. Construction invariants and immutability

An invariant is a condition that should hold whenever an object is ready for use. `AccessoryRental` requires non-null products from the same category.

Fail-fast validation rejects invalid input during construction rather than allowing a later failure in `summary()`. A thrown exception means construction did not complete normally.

Private final fields cannot be reassigned after construction. That does not automatically make every referenced object deeply immutable. The current product implementations are stateless, but the interfaces alone do not prevent future mutable implementations.

The category comparison does not select a concrete type. There is a difference between conditionals that validate data and long conditionals that replace polymorphic product selection.

9. Reading UML

- A class box gives a type's name and, optionally, its fields and operations.
- "+" means public; "-" means private.
- An interface defines a contract; an abstract class cannot be instantiated directly.
- A solid line with a hollow triangle points from a subclass to its superclass.
- A dashed line with a hollow triangle points from an implementation to its interface.
- A dashed dependency arrow points from the user toward the type it uses.
- A directed association can represent a stored reference.

- A “creates” label identifies a construction dependency.

Do not confuse Java's general has-a relationships with UML's stricter filled-diamond composition notation. The project diagrams use ordinary associations for the client's stored product references; they do not claim strict ownership semantics.

To study a diagram, first locate the client or workflow. Trace the abstractions it sees, then identify where concrete implementations are chosen.

10. Common mistakes

- Calling any method that returns an object “Factory Method” without a creator hierarchy.
- Creating two unrelated products and claiming they form a meaningful family.
- Instantiating concrete products inside the Abstract Factory client.
- Moving a large type switch into every caller.
- Claiming all if statements violate polymorphism.
- Confusing an interface variable with the concrete object it references.
- Claiming final references imply deep immutability.
- Adding patterns where direct construction is clearer.
- Claiming tests prove the design is correct for every possible future implementation.

11. Practice questions and answers

Why can Vehicle refer to a Car?

Car implements Vehicle, so it satisfies the type relationship and can be used through that interface.

Who chooses the concrete vehicle?

The selected concrete creator, through its createVehicle() override.

Who chooses which creator to use?

Main, the composition root.

Can AccessoryRental work with a new accessory implementation?

Yes, if it is supplied through AccessoryFactory and satisfies the product contracts and family invariant.

Why are there two creation methods in AccessoryFactory?

There are two distinct product types in each family: safety equipment and a security device.

What does the mixed-family test show?

The client rejects a Helmet paired with a CarAlarm because their category labels differ.

What does it not show?

It does not prove physical suitability or verify all possible factory implementations.

Where is polymorphism visible?

The overridden createVehicle() call, the factory-interface creation calls, and calls through product interfaces.

Why are we using patterns in a tiny project?

To practice the structures and tradeoffs. A similarly small production program might reasonably use direct constructors.

12. Learning exercises

1. Draw the two diagrams from memory, then compare them to docs/factory-method.png and docs/abstract-factory.png.
2. Trace one car rental without running the program. Predict which methods execute in order.
3. Add Bicycle and BicycleFactory on a practice branch. Explain why prepareRental() needs no type switch.
4. Supply a test factory returning Helmet and CarAlarm. Predict the exception before running it.
5. Explain what must change to add a third product type such as NavigationDevice.
6. Explain the difference between DIP and dependency injection in your own words.
7. Run Main.java and PatternTests.java in VS Code. Connect each test to a contract or invariant.

Suggested learning order: OOP basics, polymorphism, Factory Method, Abstract Factory, then SOLID and tradeoffs. Learn to explain the problem before memorizing class names.

Further reading

- Freeman and Robson, Head First Design Patterns, Chapter 4 (course reference).
- Robert C. Martin, Clean Code, Chapter 6 (course reference).
- Your course's Lecture 2 slides and assigned Moodle material.
- VS Code Java project setup: <https://code.visualstudio.com/docs/java/java-project>

These notes explain the submitted teaching example; use your course material for the instructor's exact terminology.